

SouvenirPOS — Implementation Explanation

Course: IT342-G01 — Systems Integration and Architecture 1

Project: SouvenirPOS – A Cross-Platform Sales Point-of-Sale System for Souvenir Shops

Prepared by: Ritchie Jay D. Erag

Core Feature: Process Sale (Record a Sale Transaction)

Repository: <https://github.com/chieeng/SouvenirPOS>

This document explains the implementation of the system's core feature — **Process Sale** — by identifying the source-code components involved, the responsibility of each, and the validation and error-handling mechanisms that keep the sale data correct and secure. The architecture follows the SRS: the Spring Boot backend is the single source of truth (NFR-011), and both the ReactJS web app and the Android Kotlin app call the same REST API.

1. Relevant Files, Classes, Functions/Methods, and Components

1.1 Backend — Spring Boot (source of truth)

Component	File	Key methods / members
SaleController <code>@RestController</code>	backend/.../sale/controller/SaleController.java	createSale(SaleCreateRequest) → POST /api/sales listSales(LocalDate) → GET /api/sales; summary() GET /api/sales/summary; getSale(Long) → GET /api/sales/{id}
SaleService <code>@Service</code>	backend/.../sale/service/SaleService.java	createSale(...) (@Transactional), listSales(date) getSale(id), summary(), private currentUser()
Sale <code>@Entity</code>	backend/.../shared/domain/Sale.java	Fields saleDateTime, cashier, totalAmount, paymentAmount, changeAmount, items; helper addItem(SaleItem)
SaleItem <code>@Entity</code>	backend/.../shared/domain/SaleItem.java	Fields category, quantity, unitPrice, subtotal
SaleRepository	backend/.../sale/repository/SaleRepository.java	save(), findById(), findAllByOrderBySaleDateTimeDesc(), findBySaleDateTimeBetweenOrderBySaleDateTimeDesc()
SaleCreateRequest (DTO)	backend/.../sale/dto/SaleCreateRequest.java	items (@NotEmpty @Valid), paymentAmount (@NotNull)
SaleItemRequest (DTO)	backend/.../sale/dto/SaleItemRequest.java	categoryId (@NotNull), quantity (@Min(1)), unitP (@NotNull @DecimalMin("0.01"))
SaleResponse / SaleItemResponse	backend/.../sale/dto/	Serialize the saved sale (id, dateTime, cashierName, totals, line it the receipt
CategoryRepository	backend/.../category/repository/CategoryRepository.java	findById(categoryId) — validates each line's category (FR
GlobalExceptionHandler <code>@RestControllerAdvice</code>	backend/.../shared/exception/GlobalExceptionHandler.java	Maps exceptions to HTTP status + JSON message
JwtAuthFilter, JwtService, SecurityConfig, CustomUserDetailsService	backend/.../shared/security/	Validate JWT on every /api/** request and load the authentic cashier

1.2 Web client — ReactJS

Component	File	Key functions / state
POSPage	web/src/pos/components/POSPage.jsx	State: categories, category, priceInput, quantity, cart, payment, receipt. Functions: handleNumpadPress(), adjustQuantity(), handleAddToCart(), handlePaymentChange(), handleCheckout()
client (Axios instance)	web/src/shared/api/client.js	baseUrl: '/api'; request interceptor adds Authorization: Bearer <token>
AuthContext	web/src/auth/context/AuthContext.jsx	login(), logout(), holds JWT + current user

1.3 Mobile client — Android Kotlin / Jetpack Compose

Component	File	Key functions / state
PosScreen (Composable)	mobile/.../pos/ui/PosScreen.kt	Three steps — Sell / Cart / Payment; composables Keypad, Stepper, CategoryChip, ReceiptDialog
PosViewModel	mobile/.../pos/viewmodel/PosViewModel.kt	State: categories, priceInput, quantity, cart (CartLine), payment. Functions: onNumpad(), incQuantity()/decQuantity(), addToCart(),

Component	File	Key functions / state
		<code>onPaymentNumpad()</code> , <code>checkout()</code> ; <code>computed total</code> , <code>change</code> , <code>canCheckout</code>
PosRepository	<code>mobile/.../pos/data/PosRepository.kt</code>	<code>categories()</code> , <code>createSale(SaleCreateRequest)</code>
ApiService (Retrofit)	<code>mobile/.../core/network/ApiService.kt</code>	<code>@POST("sales")</code> , <code>@GET("categories")</code> , etc.
AuthInterceptor	<code>mobile/.../core/network/AuthInterceptor.kt</code>	Adds the JWT header to every outgoing request

2. Responsibilities of Each Code Component

- SaleController** — Thin HTTP layer. Receives the sale request, triggers Bean Validation with `@Valid`, delegates to `SaleService`, and returns `201 Created` with the saved sale. It contains no business logic.
- SaleService.createSale()** — The heart of the feature and the authoritative business logic. It (1) resolves the authenticated cashier via `currentUser()` and attaches it to the sale (BR-004); (2) loops the request lines, loads each `Category`, and computes `subtotal = unitPrice × quantity` and the running `total` using `BigDecimal` at 2 decimals (FR-008, BR-007); (3) verifies `payment ≥ total` (BR-003) and computes `change = payment - total` (FR-009); (4) saves the whole graph in one transaction (FR-011).
- Sale / SaleItem entities** — Map the object model to the `sales` and `sale_items` tables. `Sale` owns a cascaded list of `SaleItem`s (one-to-many) and requires a non-null `cashier` (BR-004); `SaleItem` permanently stores the price entered at sale time (BR-005). Money columns are `precision 12`, `scale 2` (BR-007).
- SaleRepository** — Spring Data JPA persistence; generates INSERT/SELECT SQL and exposes the derived queries used for history and the dashboard.
- SaleCreateRequest / SaleItemRequest** — Define the request contract and carry the declarative validation rules (see §3). They decouple the API from the entities.
- SaleResponse / SaleItemResponse** — Shape the outgoing JSON used by both clients to render the digital receipt (FR-010) and history/detail views.
- Security classes** — `JwtAuthFilter` validates the bearer token on each request and populates the `SecurityContext`; `SecurityConfig` permits only `/api/auth/login` anonymously and protects everything else (BR-001, NFR-004); `JwtService` issues/parses tokens; `CustomUserDetailsService` loads users for authentication.
- POSPage (web) / PosScreen + PosViewModel (mobile)** — Presentation and interaction. They gather the cashier's input (category, price via keypad, quantity, payment), keep the cart in local UI state, do lightweight pre-validation for a smooth UX, send the checkout request, and display the returned receipt. They treat the server's computed values as authoritative.
- client / ApiService + interceptors** — Networking layer that attaches the JWT and (de)serializes JSON between the clients and the API.

3. Validation and Error-Handling Mechanisms

3.1 Client-side (usability, first line)

- Add-line guard.** A line is only added when a category is selected, the price parses to > 0 , and quantity ≥ 1 (`handleAddToCart()` / `PosViewModel.addToCart()`); otherwise a message is shown and nothing is added.
- Checkout guard.** The Checkout button is disabled until the cart is non-empty and cash tendered $\geq$ total (web: `disabled={cart.length === 0 || paymentAmount < total}`; mobile: `canCheckout`). This mirrors BR-002 and BR-003 before any request is sent.
- Input sanitisation.** The keypad and payment fields only accept digits and a single decimal point.

3.2 Server-side (authoritative, NFR-011)

- Bean Validation** (triggered by `@Valid` on the controller): `@NotEmpty` on `items` enforces "at least one line item" (BR-002); `@Min(1)` on `quantity` enforces FR-007; `@DecimalMin("0.01")` on `unitPrice` rejects zero/negative prices; `@NotNull` guards `categoryId` and `paymentAmount`.
- Business-rule checks in SaleService**: the category id is looked up and an unknown id throws an error; `payment ≥ total` is re-checked on the server (BR-003) regardless of the client; the cashier is required from the security context (BR-004).
- Recomputation, not trust.** Totals, subtotals, and change are always recomputed on the server from unit price $\times$ quantity, so tampered client totals cannot affect the stored record (FR-008/009).
- Transactional integrity.** `createSale()` is `@Transactional`; if any line fails validation or persistence, the whole sale is rolled back and nothing is saved — no partial sales (supports NFR-007).
- Immutability.** There is no update or delete endpoint for sales, so a completed sale can only be read afterwards (BR-008).

3.3 Centralised error handling — `GlobalExceptionHandler (@RestControllerAdvice)`

Exception	HTTP status	Body / meaning
<code>MethodArgumentNotValidException</code> (Bean Validation)	400 Bad Request	Map of <code>field</code> → <code>message</code> (e.g. "Quantity must be at least 1")
<code>IllegalArgumentException</code> (payment < total, category not found, no authenticated user)	400 Bad Request	<code>{ "message": "..."} — the specific reason</code>
<code>BadCredentialsException</code>	401 Unauthorized	Wrong username/password on login
<code>AccessDeniedException</code>	403 Forbidden	Non-owner attempting an owner-only action (RBAC, FR-017)

Exception	HTTP status	Body / meaning
Missing/invalid/expired JWT (Spring Security entry point)	401 Unauthorized	Unauthenticated request to a protected endpoint (BR-001, NFR-004, NFR-013)

Both clients read the returned message and surface it to the user (e.g. the web POS shows it in `error`, the mobile POS in `viewModel.error`), and the sale is **not** recorded. This gives consistent, predictable failure behaviour across web and mobile.